

Future Comparisons Between Chess Engine Calculation Algorithms

Word Count: 4985

Abstract

Chess engines are a subject with limited prior research, despite the copious amount of existing data found in engine tournaments such as TCEC and CCC. This research focused on the comparisons between two prominent chess engine calculation algorithms, Minimax and MCTS. The focus of research was further narrowed to comparisons between Stockfish and Lc0, two top chess engines which utilize Minimax and MCTS, respectively. Analysis was conducted on existing data from matches between Stockfish and Lc0 from TCEC and CCC, noting trends in their improvements over time. The results showed a sizable difference in the current strengths of Stockfish and Lc0, but the algorithms still proved to be comparable and competitive in line with past trends, even as technology improved. Limiting factors of engine strength were also noted based on correlations with nodes per second and total node values over different engine versions. It was concluded that different engine calculation algorithms will continue to be comparable and competitive with improvements in technology. However, limitations of this research due to the smaller data set mean that further research can be conducted to expand on these claims.

Introduction

Chess has existed for thousands of years, and played extensively for hundreds. Most people have at least heard of chess, many know how to play, some are able to play well, and the top players in the world are exceptionally good, able to defeat other humans with ease. But no human, not even the world champion, has truly mastered the game of chess. At the moment, even computers have not mastered chess due to its sheer complexity. Approximations of the total number of possible legal chess positions place a lower bound at the magnitude of 10^{120} , called the Shannon number, a number greater than even the number of atoms in the whole universe, around

the magnitude of 10^{80} (Editorial Staff, 2021). Knowing this, it is understandable as to why no computer has looked through all the possible positions. The closest computers have come to this feat is in endgame tablebases, capable of instantaneously returning a result obtained through best play, from any position with at most seven pieces. Current tablebases contain over 400 trillion positions, still very far off from the total legal chess positions (Fiekas, n.d.). Therefore, chess engines are still limited to providing an evaluation, a numeric value representing the win probability for each player, in a given position. The derivation of this value through a specific calculation algorithm is the focus of this research.

The history of chess engines is concentrated in the second half of the 1900s and 2000s, beginning with the idea of creating a chess engine by Alan Turing and Claude Shannon in the 1950s. The 1970s saw the creation of Minimax, a calculation algorithm that is still used to this day (Mistreaver, 2019). In 1996 and 1997, chess engines saw their first major success competitively in the match between Deep Blue, an IBM supercomputer, and Gary Kasparov, the world champion. In the first match in 1996, Kasparov proved his strength by defeating Deep Blue, but Deep Blue shocked the world with a 3.5-2.5 victory over Kasparov in the 1997 rematch. Kasparov himself was even surprised at the strength of the engine, stating in an article that “I expressed my surprise and amazement at seeing a new kind of intelligence,” and that “I found myself unprepared for what turned out to be a totally new kind of intellectual challenge” (Kasparov, 1997). This assertion is notable, implying the great magnitude of this result. Following this success, chess engine development would continue into the 2000s and 2010s, with competitive engine tournaments such as the Top Chess Engine Championship (TCEC) created since engines had far surpassed human levels (Mistreaver, 2019).

However, the most notable event was the emergence of a new evaluation technique in 2017, pioneered by the creation of AlphaZero by Deepmind. This was the first engine to deviate from the proven Minimax algorithm, instead implementing neural networks and a lesser-explored technique of Monte Carlo Tree Search (MCTS). In a 100 game match between AlphaZero and the strongest traditional chess engine, Stockfish, AlphaZero won without contest, not even losing a single game. More importantly, however, was how AlphaZero won its games. In stark contrast to traditional engines, AlphaZero played less human-like, sacrificing pieces for uncertain advantages (Herkewitz, 2020). Despite this unconventional style, further analysis of AlphaZero's play indicates that it is actually extremely dangerous, with threats that Stockfish did not recognize quickly (Strogatz, 2018). Due to this unprecedented success, many speculated that neural network based engines would come to dominate over traditional engines, but others are still skeptical. Nonetheless, AlphaZero's success prompted development of engines employing similar techniques and algorithms, including Leela Chess Zero utilizing nearly identical ideas.

In observing this development and the subsequent results, though, the predicted dominance of neural networks has yet to occur, and it is far from clear whether this will ever happen. Therefore, my research question seeks to provide clarity on the future situation: Can differing chess engine calculation algorithms give comparable and competitive results as technology keeps advancing?

The focus of this research is to explore the future possibilities of two primary calculation algorithms currently used in chess engines, Minimax and MCTS. While it is possible that new algorithms and techniques could be developed in the future, the scope of this research does not include those that are impossible to predict.

Literature Review

Because exploration of chess engines requires some background knowledge and understanding of key ideas and topics, the literature review is split into two sections. Important terms and ideas will first be explained, and then prior research in this field will be analyzed.

Background Information

The Minimax algorithm is founded on the basis of two competing players—one player seeks to maximize an evaluation, and the other seeks to minimize it. Within a minimax tree, the bottom-most layer is created first, with the final evaluations of each position. Then, the players take turns making their optimal decision, continuing this process up the tree until arriving at the root node, where the final number is what the node is evaluated as (Muens, n.d.). A simple version of a possible game tree is shown in Figure 1, though a chess game tree has significantly more nodes at each level.

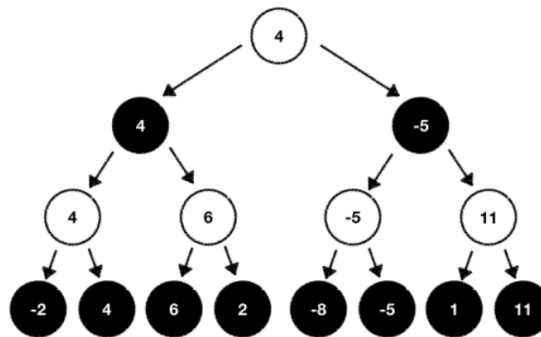


Figure 1. A simplified minimax search tree, with 2 branches per node and 4 levels. White seeks to maximize the evaluation, while black tries to minimize it (Muens, n.d.).

On its own, minimax is still quite inefficient since the total number of nodes grows exponentially. However, the real strength of minimax is obtained when combined with Alpha-Beta Pruning, significantly improving its efficiency. Alpha-Beta Pruning is a technique which eliminates entire branches of nodes that are identified as inferior even before searching,

based on the known results of adjacent nodes (Korf, 1991, pp. 99-100). The improvements of Alpha-Beta pruning are shown in Figure 2, noting nodes eliminated by Alpha-Beta pruning.

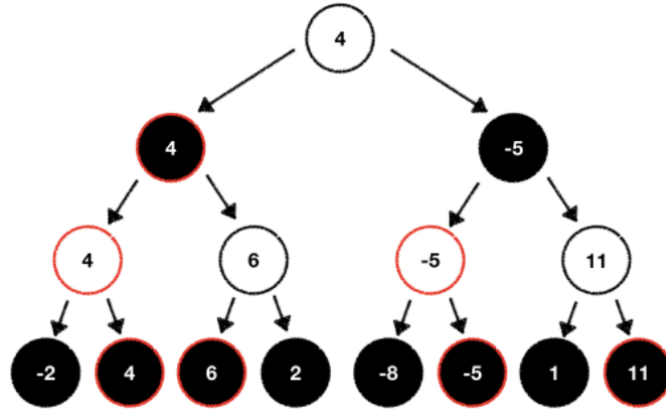


Figure 2. A minimax search tree utilizing Alpha-Beta Pruning, where only the nodes outlined in red are evaluated (Muens, n.d.).

These smaller scale representations understate the actual improvements, and a large tree containing millions of nodes experiences much more change.

In contrast, the MCTS algorithm utilizes a formula to determine the value assigned to each node, integrating the value given by the evaluation function (Browne et al., 2012, pp. 6-7). While it is not critical to understand the exact formula shown in Figure 3, the significance of each value should be noted. The main variables are the user determined constant, which changes sensitivity of the algorithm, number of times the node and parent node have been visited, and evaluation value.

$$UCT = \overline{X_j} + C \sqrt{\frac{\ln(N)}{n_j}}$$

Figure 3. The formula used to calculate node values in MCTS. X_j is the evaluation value, C is the sensitivity constant, N is the number of parent node visits, and n_j is the number of node visits (Browne et al., 2012, pp. 6-7).

These numbers are inputted into the formula which returns the node value. The game tree is somewhat similar to that of Minimax, but the overall tree shape and information contained at each node differs. A possible game tree is shown in Figure 4. (Browne et al., 2012, pp. 5-10).

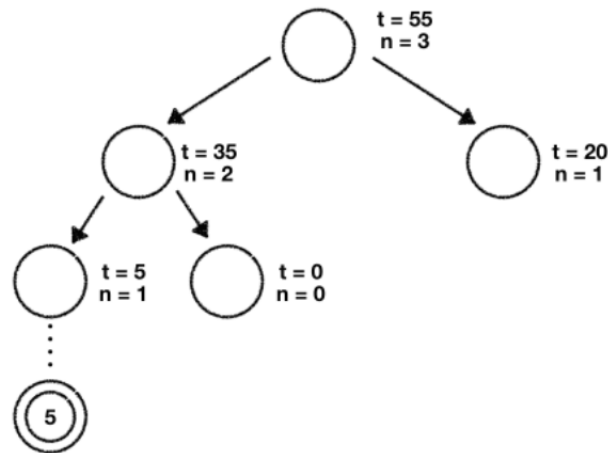


Figure 4. A MCTS search tree showing searched positions, as well as node values and visit counters (Muens, n.d.).

These two algorithms correspond directly to the two strongest chess engines currently. Stockfish, using Minimax, and Leela Chess Zero (Lc0), using MCTS, have been the dominant chess engines in recent years. While their strength and improvement are comparable, the similarities end there. Some notable differences are that Stockfish traditionally uses a handwritten evaluation function, though recent versions integrated a neural network generated function. Additionally, Stockfish runs on a Central Processing Unit (CPU), allowing it to search at the high speeds crucial for Minimax (Stockfish, 2021). On the other hand, Lc0 uses a neural network generated evaluation, fine tuning from past games. Lc0 also uses a Graphics Processing Unit (GPU), allowing it to access the large amounts of data in MCTS more efficiently (Leela Chess Zero, n.d.). The major differences between these two engines allow for further reduction of the research scope. Since these two engines likely will remain dominant in the future, and

each uses a different algorithm, comparisons between the two algorithms can be reduced to comparisons between Stockfish and Lc0.

Nodes are also an important concept in chess engines. In both Minimax and MCTS search trees, a node is essentially a possible position, whether it is one or fifteen moves in the future. Two important measures of chess engine performance, speed and total nodes, are measured in nodes per second and nodes, respectively. Accuracy of the calculation algorithm is also dependent on speed, as more positions searched decreases the likelihood of missing a critical position (Chessify, n.d.). Another idea related to nodes is depth, or the number of moves ahead the engine searches. While depth is not explored extensively in this research, it is still a very important idea in any discussion regarding chess engines.

Lastly, there are two major computer chess tournaments used in the methods of this research, TCEC and the Computer Chess Championship (CCC). These tournaments are similar in format, with the only significant difference being the time control. TCEC has longer games with around 2 hours per side, while CCC games give less than 15 minutes. Both TCEC and CCC use optimal hardware which is significantly stronger than an average computer setup. Additionally, the match format is the same with mini-matches from predetermined openings, where each computer plays one game with white and one game with black starting in a given position (Chess.com, n.d.; TCEC Chess, n.d.). These factors help minimize the impact of other variables on performance, and ensure that competition is as fair as possible. Finally, a useful feature found in both of these tournaments is the availability of statistics on engine speed and total nodes in any position during the games, as well as evaluation value graphs of the entire game. These are extremely helpful in collecting data, as described later in the methods section.

Past Research

In the field of chess engines, existing research is extremely limited, with TCEC match and season reports representing the largest source of information. These reports provide insight on the chess aspect of games—each game is analyzed for mistakes and ideas are explained (Sadler, 2022). However, the included information regarding the computing aspect is all available elsewhere and minimal explanation is given (Haworth & Hernandez, n.d.). The data shown in the reports are also the data that will be collected in the methods, so the reports offer little help.

Outside of the TCEC reports, much of the prior research in this topic primarily involves speculation, albeit educated guesses, from past trends. For example, some observers of the TCEC results and the AlphaZero vs. Stockfish match claimed that it is only a matter of time before new techniques completely overpower traditional engines (Somers, 2018). These ideas certainly have credibility—many have experience and are making predictions based on that. Nonetheless, the ideas are still not directly supported by evidence and their validity cannot be assumed.

There are, however, significant amounts of research on tangentially related topics, such as the Minimax or MCTS algorithms and computing speeds of chess engines. One example is a study conducted by Diogo Ferreira on the use of chess engine evaluations to determine the objective strength of a player. This process involves the input of a player's moves from a certain game, and based on the average evaluation decrease per move, returns a value that represents objective strength (Ferreira, n.d.). While a direct application of this is not useful, it would be interesting to apply such a process to engine games. This would only be possible with a calculating engine stronger than the inputted engines, but it could provide very helpful insight by determining an objective strength order.

Another study was conducted by Mark Levene and Judit Bar-Ilan on comparisons between engine moves. In this research, they found that different engines have preferences for different moves that are of similar evaluation, caused by minor differences in their evaluation functions (Levene & Bar-Ilan, n.d.). This result shows that despite two engines having similar identical calculation algorithms, there will always be differences that cause slightly different evaluations, and different moves are played. For an observer though, this may only seem to be a difference in playing style.

Mathieu Acher and François Esnault researched the upper bounds on engine speed, by presenting Stockfish with 270 million games to analyze. They recorded the average time it took to analyze each game, concluding that even optimal setups are unable to evaluate that many games reasonably fast, though improved technology did significantly increase speed (Acher & Esnault, 2018, p. 10). This shows that hardware and technology is a factor in computing speed, so it should be controlled in this research.

Finally, Hendrik Baier and Mark Winands studied algorithms integrating Minimax into MCTS, focusing on games simpler than chess. Although this may seem like a beneficial hybrid, combining the strength to address the shortcomings of each, the completely different approaches caused more conflict than synergy, making it much worse in practice (Baier & Winands, n.d.). Overall, these existing studies focused on a somewhat related topic, though none purely considered the topic of this research.

Because of this lack of existing research directly comparing chess engine algorithms, with even less providing concrete future predictions, this research addresses that gap in scholarly discussion. By researching the predicted future comparisons between these algorithms, the

results can either support or disprove the noted speculations, giving a more definitive analysis of the topic.

Methods

TCEC contains abundant data from matches between Stockfish and Lc0, with thousands of games and statistics available in the TCEC archives. Due to the sheer number of games, however, reduction of the data sets is necessary to provide a more manageable amount for analysis. This reduction process considered three main factors: each sample size must be large enough to show a correlation, the data sets must include different engine versions to show the change over time, and a variety of possible outcomes must be presented. With these considerations in place, the specific data set was narrowed down to three TCEC matches—TCEC Season 21 (S21) Superfinal, S17 Superfinal, and S14 Superfinal. Each of these matches was a 100 game match between Stockfish and Lc0, having a unique reason for inclusion.

The S21 Superfinal was the most recent Superfinal match between Stockfish and Lc0, giving the latest results between the two engines in a large data set. This not only allows for comparison with past results, but also shows the current status, which is important to note for extrapolation. S17 was the second and most recent season Lc0 defeated Stockfish in the Superfinal (S15 was the first). As the latest season Stockfish lost, data from this match shows how Lc0 was able to win most recently. S14 was the first season Lc0 played in the TCEC Superfinal match, shortly after its first release. Data from this season thus provides a baseline for comparisons. A notable match excluded from the data set during this process was the S15 Superfinal, but the three samples chosen maintain a consistent chronological relationship and represent a wide range of outcomes. A surprising development occurring during the research

process was the April 2022 release of a new version of Stockfish, Stockfish 15. As a result, data from matches played between Stockfish 15 and Lc0 was also appended to the data sets. This inclusion is explained further in a later section.

Within the focus data set, quantitative and qualitative data were collected. Quantitative data included the statistics on maximum speed and nodes of the engines during each game, determined from the graphs in the TCEC game windows. These numbers were copied by hand into a spreadsheet, and the average and maximum values were calculated from each set. The units for these values are important to note, since Stockfish had much larger values than Lc0. Stockfish speed was given in Mnps (millions of nodes per second), while Lc0 speed was in knps (thousands of nodes per second). Stockfish nodes were given in G (billions), while Lc0 nodes were in M (millions). Qualitative data was obtained by observing the evaluation graphs, also from the TCEC game window. Positions having significant discrepancy between the evaluations of Stockfish and Lc0 were noted, and their respective win probabilities were also included.

A general data set was also analyzed. This set contained the specific data set, as well as the excluded TCEC seasons and CCC matches between Stockfish and Lc0. For matches in this set, the total number of points was recorded for Stockfish, Lc0, and Komodo if applicable. While there was no initial intent to include Komodo, its stronger than expected performance dictated that its inclusion would be necessary.

Additionally, data from CCC matches between variations of Stockfish and Lc0, such as Stockfish NNUE and Lc0 CPU, was collected similarly to the general set. The same process was also repeated for Fischer Random matches, which randomizes the initial position to negate openings. These samples include different variables, providing further comparisons in a unique

context. For example, comparisons between Lc0 and Lc0 CPU indicate the relative strength based on hardware, and opening intuition is tested through Fischer Random.

With this method, though, there are some limitations and considerations to address. A significant note is the possible human error in manual data transfer. Utilizing a computer program to record all the numbers could solve this problem in future experiments, while also increasing the speed of data compilation and allowing for larger data sets. A second consideration within the method was the recent release of Stockfish 15, an improved version of Stockfish. The significance of this release necessitates inclusion of data for strength comparisons, yet its recency limits data availability, with a total of 12 games between Stockfish 15 and Lc0 in TCEC. Nonetheless, these games were included in a new subset, TCEC S12 Division P + Cup 9. This small sample size means that any observed trends involving the new data set should be considered carefully. Finally, the last important note is that many older matches in CCC were omitted from analysis. Including these would only add redundancy among the hundreds of matches, while providing minimal benefits.

Results

The results from this research are split into two types, quantitative and qualitative data. Beginning with the quantitative data, this can also be categorized into two sets: the general data set with overall match results, and the specific data set with speed and node values. A summary of the general set data is shown in Table 1 with the green columns denoting the victor, and red denoting the loser. There are far more victories for Stockfish than Lc0, and counting this exactly gives 15 victories for Stockfish compared to just 4 for Lc0. Komodo is also noted in 4 rows, where it placed second above Lc0. This strength gap is critical to the discussion of conclusions.

Tournament Name	Stockfish Points	Leela Points	Other
TCEC S22 Division P	37.5	34	Komodo- 37
TCEC S21 Superfinal	56	44	N/A
TCEC S20 Superfinal	53	47	N/A
TCEC S19 Superfinal	54.5	45.5	N/A
TCEC S18 Superfinal	53.5	46.5	N/A
TCEC S17 Superfinal	47.5	52.5	N/A
TCEC S16 Division P	26.5	25.5	Alliestein- 26
TCEC S15 Superfinal	46.5	53.5	N/A
TCEC S14 Superfinal	50.5	49.5	N/A
CCC 17 Rapid Semifinal	65.5	58	Komodo- 59
CCC 16 Blitz Semifinal	107	90	Komodo- 97
CCC 16 Bullet Semifinal	368	295	Komodo- 334.5
CCC 16 Rapid Finals	174	126	N/A
CCC 2021 Blitz Finals	171.5	128.5	N/A
CCC 2021 Rapid Finals	278.5	219.5	N/A
CCC 2020 Blitz Finals	277	223	N/A
Post CCC 14 Bullet Match	96.5	103.5	N/A
Post CCC 14 Blitz Match	101	99	N/A
CCC 13 Rapid Finals	94	106	N/A

Table 1. The overall match results, including points for Stockfish, Lc0, and other engines within each sample.

The data from variations, listed in Table 2, clearly show a difference in strength between versions of engines. Most notable is the score margin between Lc0 CPU and Stockfish compared with the traditional Lc0 vs Stockfish, and this level difference is also seen in the head-to-head match between Lc0 and Lc0 CPU. This observation proves the necessity for Lc0 to utilize GPU for best performance.

Variant	Stockfish Points	Leela Points	Other
SF Classic vs Lc0 CPU Blitz	146	54	N/A
SF vs Lc0 CPU Blitz	157	43	N/A
SF NNUE vs Lc0 Blitz	89.5	110.5	N/A
SF NNUE vs Lc0 CPU Blitz	128	72	N/A
TCEC S19 FRC (960) Final	29	21	N/A
CCC Chess 960 Blitz Semifinal	123.5	115	Komodo- 122.5
Lc0 vs Lc0 CPU Rapid	Lc0- 8	Lc0 CPU- 4	N/A

Table 2. The overall results from variant matches, including different versions of engines as well as Fischer Random Chess.

The other set of quantitative data included the speed and node values from the specific data set. These numbers were initially compiled into a larger spreadsheet, and a summary of the mean and maximum values in each set is shown in Table 3. Within each cell, the first number is the average value and the second is the maximum. After accounting for the units, the Stockfish and Lc0 values are not even close, with Stockfish having numbers hundreds times larger than Lc0. This supports the claim that Stockfish has a faster search method, looking through significantly more positions than Lc0.

Tournament Name	Stockfish Max Speed (Mnps)	Stockfish Max Nodes (G)	Leela Max Speed (knps)	Leela Max Nodes (M)
TCEC S22 Division P and Cup 9	118.725	21.8	396.55	86.8
	197.2	49.3	710.1	185.7
TCEC S21 Superfinal	209.947	85.379	329.697	243.672
	377.8	276.2	716.6	399.7
TCEC S17 Superfinal	272.706	79.197	1545.906	63.231
	404.2	246.1	5100	1000
TCEC S14 Superfinal	96.281	40.776	424.876	72.338
	160.3	105.4	1300	704.5

Table 3. The values of average maximum speed, absolute maximum speed, average maximum nodes, and absolute maximum nodes for Stockfish and Lc0 within the four specific samples.

Line graphs were also generated for each of the four data points—average max speed, absolute max speed, average max nodes, and absolute max nodes. These graphs are included in Figure 8. With only four data sets, and also considering the influence of other variables on these values, it is not surprising that the trends are not obvious.

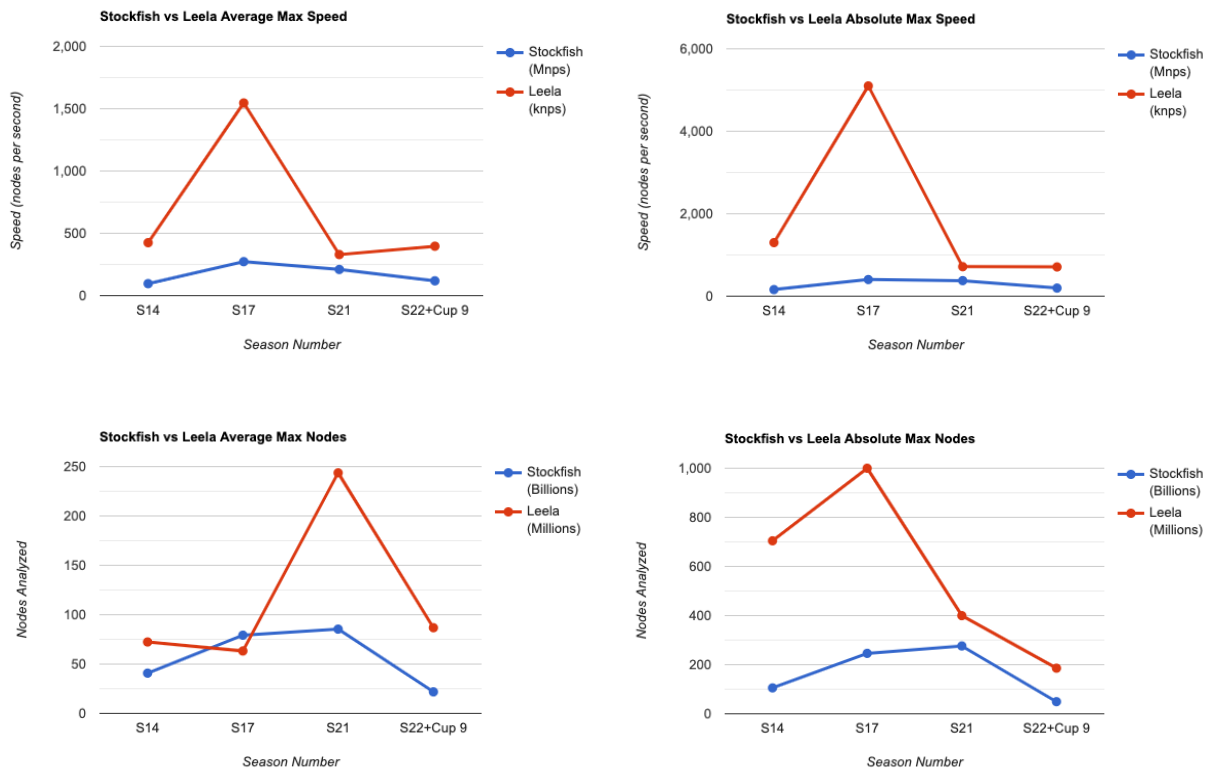


Figure 8a (top left), 8b (top right), 8c (bottom left), and 8d (bottom right). Data from Table 3 is graphed in line graphs with corresponding values, showing average maximum speed in Figure 8a, absolute maximum speed in Figure 8b, average maximum nodes in Figure 8c, and absolute maximum nodes in Figure 8d.

However, one trend should be noted for the Stockfish values: after excluding the Stockfish 15 data, the values are generally increasing together with the improving engine strength. Thus, it is a reasonable conclusion that Stockfish strength has some relation to speed and nodes. This would also make sense, considering the minimax algorithm and its necessity to search more positions. On the other hand, the data for Lc0 is all fluctuating, with essentially no correlations. Seeing this, a plausible conclusion could be that speed and nodes are not related to Lc0 strength.

There is one significant note regarding this quantitative data. Only the maximum speed and node values in each game were recorded to show a possible strength ceiling for the engine.

This idea is good in theory, but was less successful in practice. Some games involved the maximum values occurring at the very beginning, immediately following the predetermined opening position, while others had maximums occurring in positions where the result of the game was already clear. The most egregious issues involved the maximum values occurring on the very last move. This was observed in a couple Lc0 games in earlier seasons, where speed and nodes were stable at a lower level before jumping up to values hundreds times larger on the very last move. An explanation of this observation could be the game situation, with the clear result allowing Lc0 to use more time and search all the possibilities; however, other causes of this are also possible. Nonetheless, this discrepancy skewed some averages by giving an extremely high outlier, and many of the maximum values were noted in such situations. Therefore, the results should be analyzed while still keeping this consideration in mind.

The qualitative results are best explained by referring to the graphs from TCEC, from which the data was obtained. The evaluation graph from a game within each of the first three data sets (S14, S17, S21) is shown in Figure 9, and changes in the black Lc0 evaluation should be noted.

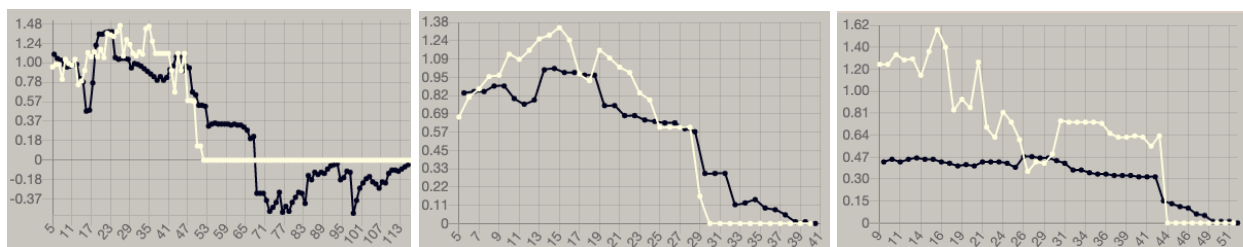


Figure 9a (left), 9b (middle) and 9c (right). Evaluation graphs from three different TCEC Superfinal games are shown, with Stockfish as white and Lc0 as black in all three games. Figure 9a is from S14, Figure 9b is from S17, and Figure 9c is from S21.

In the first graph, fluctuations in the evaluation are obvious and frequent throughout the entire game. Looking at the second graph, there are noticeably fewer fluctuations. Finally, the last graph shows almost no fluctuation and a steady evaluation. This change over time indicates

that Lc0 has tuned its calculation algorithm as it improved in strength, allowing it to stay more consistent in the latest versions. On the other hand, the fluctuation in the white Stockfish evaluation seems to have remained similar. This shows that the Stockfish evaluation function is still not as consistent as Lc0, compensating for this with the sheer number of positions it searches.

The other situation examined was in decisive games with a steadily increasing evaluation. An example of this is shown in Figure 10, a game where Stockfish beat Lc0 with white. At move 34 in the graph, when there is a large gap in the evaluations, the evaluation numbers and win probabilities were compared.

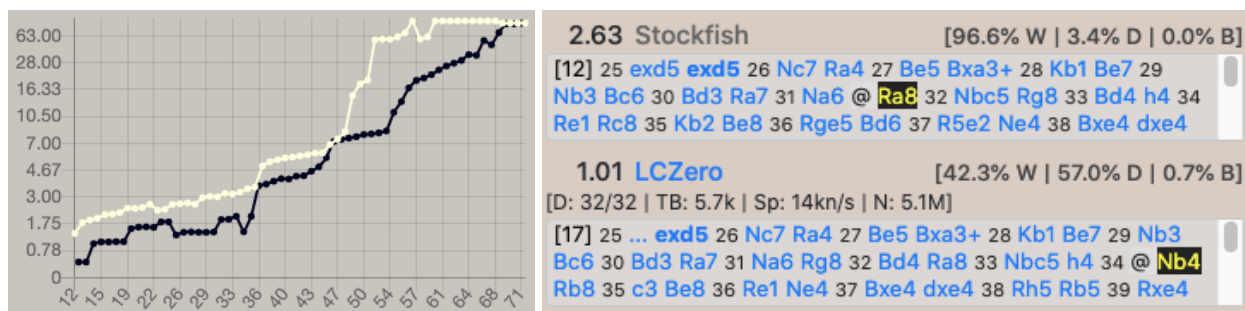


Figure 10a (left) and 10b (right). Figure 10a shows the evaluation graph where Stockfish (white) recognized a decisive advantage before Lc0 (black). Figure 10b shows the evaluation panel in a critical position on move 34, when the two algorithms give very different results.

In that position, Stockfish evaluated the position at +2.63 with a 96.6% chance that white wins. In contrast, Lc0 gave a more modest +1.01 and 42.3% chance of white winning. Just a few moves later, however, Lc0 recognized the winning advantage for white and its evaluation jumped up correspondingly. This is a notable result, showing an instance where Stockfish proved it had a stronger algorithm, seeing the win before Lc0. Similar games also occurred with the roles reversed. This shows that the two algorithms have their own strengths and weaknesses, but are relatively close in overall strength.

Discussion

Based on the results of this new research and findings from existing research, claims can be made regarding the initial research question. The data shows a clear ranking of current engine strength, giving context for the situation before considering advancements in technology. Additionally, extrapolating trends from the results can predict how changes might occur in the future, and the data also alluded to some limiting factors that should be considered for each engine.

The overall results from the general data set show a significant gap between Stockfish and other engines, evidenced by the greater number of matches and tournaments won by Stockfish compared to both Lc0 and Komodo. Therefore, the current ranking places Stockfish in the top tier, with Lc0 and Komodo in the second tier. While Lc0 certainly has seen better results than Komodo in past TCEC seasons, the strength of Komodo in CCC and its success in TCEC Season 22 make separating them difficult. It should be noted, though, that the matches between Stockfish and Lc0 are still competitive. This can be observed within the mini-matches that comprise the TCEC Superfinal, with Lc0 winning by defeating Stockfish with white and holding a draw with black from the same starting position. However, the gap between the top three and other engines is larger. In most of the events analyzed, these three engines placed as the top three with little threat from other engines. Overall, this result shows that, barring the emergence of a new engine, attention should remain on these three engines, specifically on whether Stockfish can expand its advantage.

The general set data also showed steady overall trends, so it is possible to extrapolate those trends into the future. The results between Stockfish and Lc0 clearly show both engines improving at similar rates even when considering technology, with consistent match results over

time. This alludes to the fact that improvement of algorithms and efficiency by the developers has a greater impact on strength than technology and hardware. Considering that computing speeds should continue to grow exponentially, it is reasonable to conclude that hardware will not represent a future limiting factor for these algorithms.

Knowing this, one might wonder what limiting factors could be instead. A limiting factor can be attributed to the most important factor in determining engine strength. The speed and nodes were shown to correlate with engine strength for Stockfish, especially in earlier versions. This shows that future versions of Stockfish may also experience this limitation. However, the recent introduction of neural network evaluations in Stockfish 14 and 15 adds uncertainty to this claim. Speed and node values have seen decreases following this change, since the new evaluation function needs more time to provide its stronger evaluation. This integration of neural networks does not change Stockfish's use of minimax; however, this reduction in speed justifies questioning of a correlation between speed and neural networks, and if the decrease was purely coincidental.

Lc0 has a more obvious limiting factor in its evaluation function. Because it is less dependent on speed than Stockfish, the evaluation function must be very accurate to compensate for the significantly less positions that are searched. Based on the nature of MCTS, a stronger evaluation algorithm causes the search tree to visit promising positions more, giving more opportunity to find the best moves. This is less effective with a weaker evaluation algorithm, as a lower evaluation may not warrant as deep of a search by MCTS in a position, missing critical moves. However, this lower speed also gives rise to a potential avenue of development—how strong would Lc0 be with the speeds of Stockfish? Due to its comparative novelty, Lc0 seems to

have greater potential strength, in contrast with Stockfish where exploration and development had been exhausted in many areas.

Conclusion

It is possible to revisit the research question after considering the results. The data shows that Stockfish and Lc0 provide comparable and competitive results in their matches. Even as technology advances, it is reasonable to conclude that the different algorithms will remain comparable and competitive. Past trends show that improving technology has not favored one algorithm over the other.

It is important to acknowledge limitations of this research when examining conclusions. As with any research predicting future developments, it is impossible to predict anything with complete certainty, regardless of prior trends. The process of extrapolating trends from past results is a good method to make a justified prediction, but still is imperfect. Another limitation is the comparatively small data set. Considering that there are many more games between Stockfish and Lc0, and even more with other engines, the specific sample of hundreds of games is extremely limited. While the samples were selected to highlight a broad range of possibilities, it is not possible to encompass everything within such a small data set. Thus, results from this research could differ from results obtained from larger data sets.

There are many directions for future research. The constant development of engines and gameplay in TCEC and CCC will continually increase available data. Thus, one possible direction involves including the trends from newer games, and seeing if the correlation continues. Another direction could use larger data sets that were not analyzed thoroughly in this research and also consider different engines. The sheer amount of available data means that, by

using a computer program to scan through the games, trends could be observed quickly from huge samples of games. These trends could include ones unnoticed from the smaller sets, and conclusions can be verified on a larger scale. Including other engines, such as Komodo, can also show how this data compares with different engines implementing similar algorithms. Such directions of future research will expand on the conclusions from this limited research.

Finally, real world implications of this research should be noted. While chess engines have little impact on society, their role in the chess world is significant. The way newer MCTS and neural network engines play chess is vastly different from human styles, and top players have drawn inspiration from engine ideas. Opening novelties are a primary example of this engine influence, but attacking ideas found in top games also have strong ties to engine ideas. Such a change is notable in the chess world, completely changing top level chess with engine influences in almost every aspect of the game. Impacts are also observed in the computer programming and game theory world, since results show that MCTS is as viable as Minimax for game engines. Thus, future game engine development should include this new method, in addition to traditional methods. Despite its specificity, further exploration of chess engines has significant impacts on ideas both in the chess world, and out of it.

References

- Acher, M., & Esnault, F. (2018, September 11). *Large-scale analysis of chess games with chess engines: A preliminary report*. <https://arxiv.org/pdf/1607.04186.pdf>
- Baier, H., & Winands, M. H.M. (n.d.). *Monte-Carlo tree search and minimax hybrids*.
<https://dke.maastrichtuniversity.nl/m.winands/documents/paper%2049.pdf>
- Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P.,
Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of monte carlo tree
search methods. *IEEE Transactions on Computational Intelligence and AI in Games*,
4(1), 1-43. <https://doi.org/10.1109/TCIAIG.2012.2186810>
- Chess.com. (n.d.). *Computer Chess Championship*. Retrieved April 27, 2022, from
<https://www.chess.com/computer-chess-championship>
- Chessify. (n.d.). *NPS - what are the "Nodes per second" in chess engine analysis*.
<https://chessify.me/blog/nps-what-are-the-nodes-per-second-in-chess-engine-analysis>
- Editorial Staff. (2021, August 8). *Shannon Number: What is the Shannon Number in chess?* The
Chess Journal. <https://www.chessjournal.com/shannon-number>
- Ferreira, D. R. (n.d.). *Determining the strength of chess players based on actual play*.
<https://web.tecnico.ulisboa.pt/diogo.ferreira/papers/ferreira12strength.pdf>
- Fiekas, N. (n.d.). *Syzygy Endgame tablebases*. Syzygy endgame tablebases.
<https://syzygy-tables.info>
- Haworth, G., & Hernandez, N. (n.d.). *The 20th Top Chess Engine Championship: TCEC 20*.
Retrieved May 2, 2022, from https://tcec-chess.com/articles/TCEC_20.pdf
- Herkewitz, W. (2020). AI Is Now the Undisputed Champion of Computer Chess. *Popular
Mechanics*, 18–19.

Kasparov, G. (1997). 'IBM owes mankind a rematch.'. *TIME Magazine*, 149(21), 66.

Korf, R. E. (1991). Multi-player alpha-beta pruning. *Artificial Intelligence*, 48(1), 99-111.

[https://doi.org/10.1016/0004-3702\(91\)90082-U](https://doi.org/10.1016/0004-3702(91)90082-U)

Leela Chess Zero. Lc0. (n.d.). Retrieved April 27, 2022, from <https://lczero.org/>

Levene, M., & Bar-Ilan, J. (n.d.). *Comparing move choices of chess search engines*.

<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.65.4502&rep=rep1&type=pdf>

Mistreaver. (2019, January 23). History of Chess Computer Engines. Chessentials.

<https://chessentials.com/history-of-chess-computer-engines>

Muens, P. (n.d.). Minimax and Monte Carlo Tree Search. The Intersection. Retrieved April 27,

2022, from <https://philippmuens.com/minimax-and-mcts>

Sadler, M. (2022, April 26). *TCEC Season 21 SuperFinal review* (S. Kiminki, Ed.).

https://tcec-chess.com/articles/Sufi_21_-_Sadler.pdf

Somers, J. (2018, December 28). *How the artificial-intelligence program AlphaZero mastered its games*. The New Yorker.

<https://www.newyorker.com/science/elements/how-the-artificial-intelligence-program-alpha-zero-mastered-its-games>

Stockfish. (2021). Stockfish. <https://stockfishchess.org>

Strogatz, S. (2018, December 26). *One giant step for a chess-playing machine*. The New York

Times. <https://www.nytimes.com/2018/12/26/science/chess-artificial-intelligence.html>

TCEC Chess. (n.d.). Retrieved April 27, 2022, from <https://tcec-chess.com>